

# ASTER: Adaptive Dynamic Layer-Skipping for Efficient Transformer Inference via Markov Decision Process

Fangxin Liu  
liufangxin@sjtu.edu.cn  
School of Computer Science,  
Shanghai Jiao Tong University  
Shanghai, China  
Shanghai Qi Zhi Institute  
Shanghai, China

Zongwu Wang  
wangzongwu@sjtu.edu.cn  
School of Computer Science,  
Shanghai Jiao Tong University  
Shanghai, China  
Shanghai Qi Zhi Institute  
Shanghai, China

Junjie Wang  
202215056@stu.neuq.edu.cn  
Northeastern University at  
Qinhuangdao  
Qinhuangdao, Hebei, China

Junping Zhao  
junping.zjp@antgroup.com  
AntGroup Corp.  
Hangzhou, Zhejiang, China

Haibing Guan  
hbguan@sjtu.edu.cn  
School of Computer Science,  
Shanghai Jiao Tong University  
Shanghai, China

Ning Yang  
yn937391832@sjtu.edu.cn  
School of Computer Science,  
Shanghai Jiao Tong University  
Shanghai, China

Li Jiang  
ljiang\_cs@sjtu.edu.cn  
School of Computer Science,  
Shanghai Jiao Tong University  
Shanghai, China  
Shanghai Qi Zhi Institute  
Shanghai, China

## Abstract

Transformer-based models have demonstrated remarkable performance in computer vision tasks. However, their increasing model size leads to substantial memory demands and higher latency, hindering practical deployment. This paper presents an adaptive dynamic layer-skipping framework based on Markov Decision Process, which determines optimal computational paths based on the current state of input samples. We introduce a Temporal Importance Difference Reward mechanism to address the credit assignment problem in layer-skipping decisions, and develop a knowledge distillation strategy using learnable cognitive tokens to compensate for information loss. Experiments on various models demonstrate that

our method significantly reduces computational costs while maintaining accuracy, offering a practical solution for deploying high-performance Transformer models in resource-constrained environments. The code is available at <https://github.com/wjjkhl/ASTER>

## CCS Concepts

• **Theory of computation** → **Computational complexity and cryptography**.

## Keywords

Transformer-based Model, Layer Skipping, Knowledge Distillation

## ACM Reference Format:

Fangxin Liu, Junjie Wang, Ning Yang, Zongwu Wang, Junping Zhao, Li Jiang, and Haibing Guan. 2025. ASTER: Adaptive Dynamic Layer-Skipping for Efficient Transformer Inference via Markov Decision Process. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, October 27–31, 2025, Dublin, Ireland. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3746027.3755526>

## 1 Introduction

Large Language Models (LLMs) [11] and Vision Transformers (ViTs) [5] have achieved amazing results in natural language processing and computer vision. However, the increasing scale and complexity of Transformer models present significant challenges to their widespread adoption. One of the most pressing issues is the substantial computational cost and extended inference time, which impedes their deployment in resource-constrained environments and real-time applications. Therefore, exploring efficient methods

---

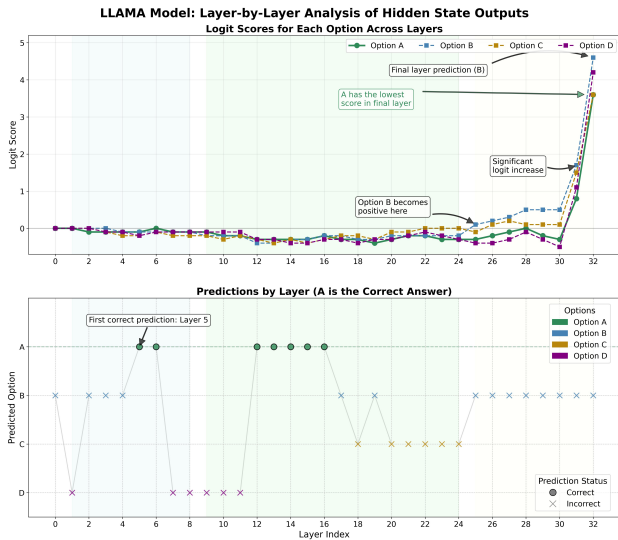
Fangxin Liu and Junjie Wang are co-first authors.  
Fangxin Liu and Li Jiang are the corresponding authors.  
This work was done when Junjie Wang was an intern at Shanghai Jiao Tong University.  
This work is supported by the National Key Research and Development Program of China (2024YFE0204300), the National Natural Science Foundation of China (Grant No.62402311), and Natural Science Foundation of Shanghai (Grant No.24ZR1433700).  
This work is also sponsored by CAAI-Ant Group Research Fund.

---

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

MM '25, Dublin, Ireland

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 979-8-4007-2035-2/2025/10  
<https://doi.org/10.1145/3746027.3755526>



**Figure 1: Layer-by-Layer Analysis of LLAMA Model’s Hidden State Outputs.** The top figure shows the logit scores for each option (A-D) across all 32 layers of the LLAMA3.1-8B model during the MMLU question-answering task, with 0 representing the initial state. The bottom figure tracks the predicted answers at each layer, illustrating that the model correctly identified option A in layers 5-6 and 12-16 but ultimately chose the incorrect option B in the final layer.

to accelerate the inference process of Transformer models is of critical importance.

To address these challenges, model compression has emerged as a promising solution to accelerate inference and reduce energy consumption. Common compression techniques such as quantization and sparsification (e.g., weight pruning and sparse attention) reduce the model size or computational complexity by lowering precision or removing unimportant components. However, finding a favorable trade-off between model accuracy and hardware efficiency remains non-trivial.

Layer skipping has recently gained attention as an alternative approach for accelerating inference. As illustrated in Figure 1, not all layers in a Transformer are equally necessary for every input. In some cases, early layers may already produce sufficient information, while further computation could introduce noise or degrade performance. Several layer-skipping methods have been proposed, including early exit strategies (e.g., SkipDecode, LayerSkip), periodic skipping, and similarity-based static skipping (e.g., FFN-SkipLLM, AdaSkip).

However, these approaches suffer from key limitations. In practical inference scenarios, optimal layer selection is highly input- and task-dependent, yet most existing methods skip pre-defined layers, lacking adaptability to dynamic inputs. Furthermore, aggressive layer skipping may lead to information loss or semantic drift, resulting in significant accuracy degradation. As a result, existing strategies are often limited in how many layers they can effectively skip without compromising performance.

To overcome these limitations, we propose an adaptive and dynamic layer-skipping strategy that flexibly selects which layers to retain for each input, and incorporates an information compensation mechanism to mitigate the potential accuracy drop. Inspired by the human decision-making process—where actions are guided by contextual understanding and adaptive reasoning—we formulate layer skipping as a Markov Decision Process (MDP), allowing the model to determine its computational path based on intermediate states. To enhance the capacity of the model to preserve semantic integrity despite skipping, we introduce a guiding token, inspired by the [CLS] token in ViTs. This token helps distill critical information from a teacher model that performs full-layer computation, enabling the student model to better approximate the teacher’s output even with fewer active layers. Additionally, our framework is orthogonal and compatible with other compression techniques such as quantization and sparsity, offering further acceleration potential. In summary, our contributions are as follows:

- We model adaptive layer skipping as a Markov Decision Process (MDP) and design a dynamic scoring mechanism that determines which layers to skip and when to exit based on token-level representations, offering a flexible and context-sensitive skipping strategy.
- We introduce a specialized learnable token that facilitates knowledge distillation from a full-layer teacher model, compensating for information loss during skipping. This enables a higher skip rate without compromising model accuracy, thereby improving overall inference efficiency.
- We conduct extensive experiments across various Transformer architectures and data modalities, showing that our approach significantly enhances inference speed with minimal accuracy loss, providing a practical solution for deploying large models in resource-constrained environments.

## 2 Related Works & Motivation

In this section, we first review existing layer skipping and knowledge distillation techniques, and discuss the importance of different layers. We then analyze the importance distribution across layers and sublayers in various models, revealing the underlying factors that enable layer skipping and highlighting the shortcomings of current approaches. These insights motivate our design principles.

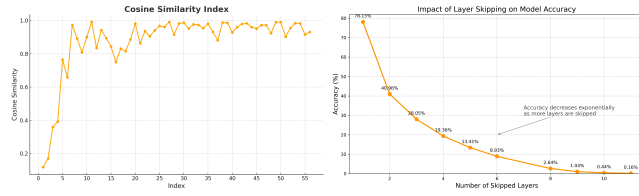
### 2.1 Dynamic Layer Skipping

The increasing computational cost of LLMs has driven extensive research into efficient inference strategies. Among them, dynamic layer skipping has emerged as a promising approach that reduces computation by skipping redundant layers while maintaining model performance. The field has evolved from simple static heuristics to more sophisticated, adaptive methods guided by input semantics. Early efforts focused on static strategies such as LayerDrop [7], which randomly drops layers during training to regularize models. Although effective for compression, such methods lacked input-awareness. SkipDecode [2] later extended this idea to autoregressive decoding by incorporating batching and caching mechanisms. However, partial forward passes posed key-value cache consistency challenges, limiting scalability. To better align computation with

input complexity, early-exit strategies emerged, allowing predictions at intermediate layers when sufficient confidence is achieved. PABEE [25] first introduced this idea in BERT, and CALM [17] extended it to encoder-decoder models with confidence-aware mechanisms. These works revealed that many inputs do not require full model depth, particularly in simpler tasks. However, early-exit decisions are often based on shallow indicators, limiting robustness in ambiguous or out-of-distribution inputs.

Recent advances push toward more fine-grained and adaptive control. ShortGPT [14] evaluates token-layer similarity to skip redundant computations, showing that many LLM layers exhibit surprising redundancy. AdaSkip [9] further refines this idea by operating at the sublayer level—selectively skipping attention blocks in long-context scenarios based on input-output similarity. These methods demonstrate the benefits of token-level adaptivity but still rely on pre-defined similarity metrics.

Some recent works shift focus to component-level optimization. FFN-SkipLLM [12] selectively skips feed-forward blocks—responsible for two-thirds of parameters—while preserving attention paths to avoid disrupting key-value caches. LayerSkip [6] combines early exit with self-speculative decoding for autoregressive tasks, eliminating the need for auxiliary models. AdaInfer [8] takes a statistical approach by predicting per-input early stopping using classical classifiers, achieving computation savings with minimal quality drop.



**Figure 2: Layer Skipping Effects: Cosine Similarity and Model Accuracy Analysis.** The left graph shows the cosine similarity across layer indices, illustrating that representation similarity stabilizes after the early layers despite initial fluctuations. The right graph displays the exponential decline in model accuracy as more layers are skipped, highlighting that minimal skipping maintains reasonable performance, whereas excessive skipping leads to a significant accuracy drop.

## 2.2 Knowledge Distillation

Knowledge distillation has emerged as a promising technique for transferring knowledge from large, over-parameterized models to smaller, more efficient ones. It began with soft-target matching, but has since evolved into multi-stage, task-specific, and collaborative frameworks. Hinton et al. [10] laid the groundwork by introducing soft-target distillation, where student models are trained to match teacher output distributions. DistilBERT [15] demonstrated that such distilled models could achieve comparable performance with significantly fewer parameters, establishing the method’s relevance in transformer architectures. Later, architectural distillation extended beyond outputs to internal structures. TinyBERT [13], MobileBERT [20], and MINILM [23] introduced frameworks for

transferring embedding, attention, and intermediate layer knowledge. These approaches enhanced transfer efficiency, particularly under resource constraints. Recent innovations include collaborative and self-distillation. MT-BERT [24] aggregates knowledge from multiple domain-specific teachers, while BERT-PKD [19] and Ensemble Distillation [1] exploit intermediate representations and multiple perspectives for robust transfer. These collaborative frameworks improve generalization and robustness across tasks.

## 2.3 Motivation

Many Transformer-based models exhibit significant redundancy across layers, as shown by recent work like ShortGPT [14] and AdaSkip [9]. These models can often skip a substantial portion of computations without noticeably harming performance. While layer skipping techniques—such as early exit [6] and adaptive skipping [9]—have made notable progress, most of them face challenges in balancing skip ratio and prediction quality, especially under highly dynamic input conditions. Some rely on shallow features or static heuristics, limiting their adaptability across modalities. Knowledge distillation has proven effective for model compression, enabling compact models to retain the capabilities of their larger teachers [10, 13, 23]. However, its potential for compensating information loss during runtime layer skipping remains underexplored.

To better understand the limitations and potential of existing skipping strategies, we conduct a layer-wise analysis of representation similarity and accuracy degradation under aggressive skipping. As shown in Figure 2 (left), a substantial portion of Transformer layers exhibit high similarity between their input and output activations (excluding bias), indicating low information gain. This suggests that skipping such layers could remove redundant computation with minimal impact on performance, which partially explains why many layer-skipping methods remain effective.

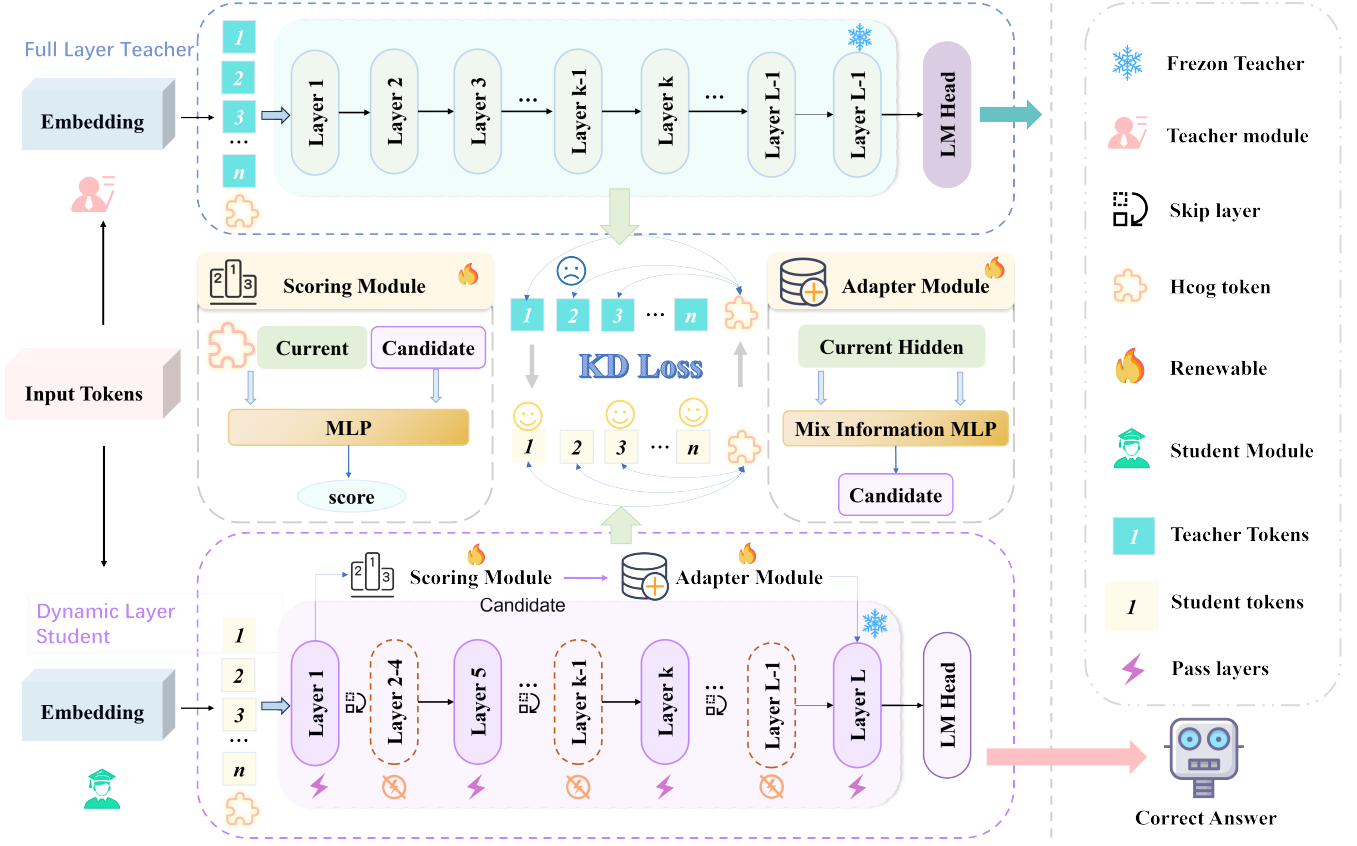
However, we also observe a critical limitation in existing *aggressive & fixed skipping* schemes. As shown in Figure 2 (right), skipping more than 4 layers in a 12-layer model (e.g., ViT) leads to a sharp drop in performance. This highlights that although redundancy exists, the skipping budget is inherently limited: naive static skipping or skip-maximization strategies offer only marginal speedup due to this upper bound.

**Key Insight:** We posit that dynamic layer skipping can be greatly improved by modeling it as a sequential decision-making process, guided by token-level representations. Furthermore, incorporating distillation-informed signals during inference can help mitigate the accuracy degradation caused by missing computations.

These insights motivate us to develop a unified framework that (1) learns to adaptively skip layers through a learned policy informed by sequential input representations, (2) Through a specialized cognitive token, it absorbs missing knowledge via online distillation from the full-layer teacher model. Our approach aims to improve inference efficiency while maintaining model performance across a range of tasks and modalities.

## 3 Method

To tackle the inefficiency and rigidity of fixed layer-wise execution in Transformers, we propose ASTER, an adaptive dynamic layer-skipping framework (Figure 3) that selects computation paths



**Figure 3: Overview of the ASTER Framework.** The method begins by processing the embedding layer and the first Transformer layer. At each decision point, a scoring model evaluates potential layer-skipping paths based on the current representations. When layers are skipped, dynamic feature representations are introduced to compensate for the missing computations. The entire system is trained using a dual-matching knowledge distillation mechanism and optimized with a Temporal Importance Difference Reward strategy, effectively balancing inference speed and model accuracy.

based on input-specific representations. Specifically, ASTER employs a scoring model to evaluate the importance of each layer during inference and dynamically decides whether to skip them. To prevent overly aggressive skipping, a minimum path length constraint is introduced, with a penalty applied when the path falls below the threshold. Additionally, ASTER integrates dynamic feature representations to compensate for skipped layers, preserving model expressiveness. The entire framework is trained with a dual-matching knowledge distillation mechanism and optimized using a Temporal Importance Difference Reward strategy. By combining these techniques, ASTER effectively reduces redundant computation while maintaining accuracy.

### 3.1 Problem Definition

Given a pre-trained language model  $\mathcal{M}$  consisting of  $L$  Transformer layers, for an input sequence  $\mathbf{X}$ , traditional inference requires sequential computation of all  $L$  layers with a computational complexity of  $O(L \cdot n^2)$ .

The **Dynamic Layer Skipping** aims to construct a decision system  $\mathcal{D}$  that determines the optimal computation path  $\mathcal{P} =$

$\{(l_1, l_i), (l_i, l_j), \dots, (l_j, L)\}$  for each input  $\mathbf{X}$ , where  $(l_i, l_j)$  represents direct computation from layer  $l_i$  to layer  $l_j$  (skipping intermediate layers),  $k$  is the number of decision steps, and  $1 \leq l_1 < l_i < \dots < l_j < L$ .

Let  $f_{\mathcal{M}}(\mathbf{X})$  denote the output of the complete model on input  $\mathbf{X}$ , and  $f_{\mathcal{M}, \mathcal{P}}(\mathbf{X})$  denote the model output following computation path  $\mathcal{P}$ . Our objective is to minimize the average number of computed layers  $\bar{L}$  while ensuring that the performance degradation remains within an acceptable range:

$$\min_{\mathcal{D}} \mathbb{E}_{\mathbf{X}}[|\mathcal{P}_{\mathbf{X}}|] \quad \text{s.t.} \quad \mathbb{E}_{\mathbf{X}}[d(f_{\mathcal{M}}(\mathbf{X}), f_{\mathcal{M}, \mathcal{P}_{\mathbf{X}}}(\mathbf{X}))] \leq \epsilon \quad (1)$$

where  $|\mathcal{P}_{\mathbf{X}}|$  is the number of layers in path  $\mathcal{P}_{\mathbf{X}}$ ,  $d(\cdot, \cdot)$  is the accuracy difference, and  $\epsilon$  is the tolerance threshold for performance degradation.

In practice, we reformulate this problem as finding the optimal policy function  $\pi_{\theta}$  that selects the next layer  $l_{t+1}$  at each decision step  $t$ , based on the current layer  $l_t$  and model state  $h_t$ :

$$l_{t+1} \sim \pi_{\theta}(l_{t+1}|l_t, h_t) \quad (2)$$

where  $\mathbf{h}_t$  contains the hidden state information of the current layer, particularly the representation of the cognitive token, and  $\theta$  represents the policy parameters.

Ultimately, our goal is to find parameters  $\theta^*$  such that:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\mathbf{X}} [(l_{t+1} - l_t)] \quad \text{s.t.} \quad \mathcal{L}(\pi_{\theta}) - \mathcal{L}(\mathcal{M}) \leq \epsilon \quad (3)$$

where  $\mathcal{L}(\cdot)$  is the loss function on the downstream task.

An LLM-based multi-agent reinforcement learning framework for an MDP, denoted as  $\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}$ .

**State ( $\mathcal{S}$ ):** In the current step  $t$ , the state  $s_t \in \mathcal{S}$  is represented by a combination of historical states, denoted as  $s_t = (h_t, l_t)$ , where  $h_t \in \mathbb{R}^{B \times N \times D}$  is a one-dimensional array of historical observation sequences (of dimension  $B \times N \times D$ ),  $N$  for the number of historical states,  $D$  for the dimension of historical observations, and  $l_t \in \{0, 1, \dots, L-1\}$  is the historical index of the current step (with  $L$  representing the total number of historical steps).

**Action ( $\mathcal{A}$ ):** At time  $a_t \in \mathcal{A}$  is an individual action, denoted as  $C_t \subset \{l_t + 1, \dots, L-1\}$  among  $L-1$  possible action, where the set  $C_t$  is the set of all feasible actions at time  $t$ .

**Policy ( $\pi_{\theta}$ ):** The policy  $\pi_{\theta}$  is a learned policy that maps the state  $s_t$  to the action  $a_t$ , denoted as  $\pi_{\theta}(a_t | s_t)$ , where the optimal policy is selected based on the maximum expected cumulative reward over all possible actions.

**Score ( $\mathcal{S}$ ):** A score function that evaluates the performance of the policy, denoted as  $S(\text{Scorer})$ , which is a real-valued function.

**Reward ( $\mathcal{R}$ ):** In each step, the reward  $r_t$  is a function of the state  $s_t$  and action  $a_t$ , denoted as  $\mathcal{R}(s_t, a_t)$ . The reward is designed to guide the agent's learning process toward maximizing the cumulative reward.

**Transition ( $\mathcal{P}$ ):** The transition probability from state  $s_t = (h_t, l_t)$  to state  $s_{t+1} = (h_{t+1}, l_{t+1})$  is denoted as  $\mathcal{P}(s_{t+1} | s_t, a_t)$ , where  $h_{t+1}$  and  $l_{t+1}$  represent the next state's historical observations and index, respectively.

### 3.2 Cognition-Enhanced Layer Representation

Traditional LLM architectures lack an explicit representation of the current computational state, making dynamic layer decisions difficult. Inspired by the design of the CLS token in the ViT model, to address this issue, we utilize the last token of the model's input sequence as this special cognitive token <cognitive>:

$$\mathbf{X}' = \{x_1, x_2, \dots, x_{n-1}, \text{<cognitive>}\} \quad (4)$$

The hidden representation of this token at each layer naturally captures the current computational state information, which we define as:

$$h_{\text{cog}}^{(l)} = \text{Transformer}_l(\mathbf{X}')[\text{<cognitive>}] \quad (5)$$

For models like ViT and BERT that naturally incorporate a [CLS] token, we utilize the [CLS] token as this special cognitive token, whereas for "Decoder-Only" models such as LLaMA, we employ the last token of the model's input sequence as this special cognitive token.

This design offers three key advantages: (1) it requires no modifications to the underlying model architecture; (2) the cognitive token is updated at each layer, reflecting accumulated information;

and (3) the token position is fixed, facilitating easy extraction and utilization.

### 3.3 Layer Skipping Decision Mechanism

Given the current layer  $l_{\text{curr}}$  of the model, our objective is to determine the next layer  $l_{\text{next}}$  to compute, where  $l_{\text{next}} \in \{l_{\text{curr}} + 1, l_{\text{curr}} + 2, \dots, L\}$ , including the termination option ( $l_{\text{next}} = L$ ).

The layer skipping policy is implemented through a scoring model  $S_{\theta}$  that calculates the skipping decision score based on the cognitive representation, current layer position, and candidate layer position:

$$s = S_{\theta}(h_{\text{cog}}, l_{\text{curr}}, l_{\text{cand}}) \quad (6)$$

Specifically implemented as:

$$S_{\theta} = \text{Softmax}(\text{MLP}([h_{\text{cog}}; E(l_{\text{curr}})]; E(l_{\text{cand}})])) \quad (7)$$

where  $E(\cdot)$  is the layer position embedding function,  $[\cdot; \cdot]$  denotes vector concatenation, and the MLP structure is:

$$\text{MLP}(x) = W_2 \cdot \text{GELU}(W_1 \cdot x + b_1) + b_2 \quad (8)$$

During training, we employ a temperature-based randomization strategy to balance exploration and exploitation:

$$P(l_{\text{next}} = l | h_{\text{cog}}, l_{\text{curr}}) \propto \exp\left(\frac{S_{\theta}(h_{\text{cog}}, l_{\text{curr}}, l)}{\tau}\right) \quad (9)$$

where  $\tau$  is the temperature parameter that gradually decreases as training progresses:  $\tau_t = \tau_0 \cdot e^{-\alpha t}$ .

During inference, we use a greedy strategy to select the layer with the highest score:

$$l_{\text{next}} = \arg \max_{l \in \{l_{\text{curr}} + 1, \dots, L\}} S_{\theta}(h_{\text{cog}}, l_{\text{curr}}, l) \quad (10)$$

### 3.4 Dynamic Adapter

When the scorer selects a non-consecutive layer (i.e.,  $l_{t+1} > l_t + 1$ ), directly using the output  $h_t$  from layer  $l_t$  as the input for layer  $l_{t+1}$  can lead to performance degradation due to a mismatch in representation spaces. The role of the Dynamic Adapter is to address this issue.

The adapter is a lightweight module that learns a function  $f_{\text{adapter}} : \mathbb{R}^D \rightarrow \mathbb{R}^D$ , which smoothly transitions the hidden state from the context of the source layer  $l_t$  to that of the target layer  $l_{t+1}$ . Its design integrates positional information and residual connections.

First, we compute a "skip signal"  $\mathbf{v}_{\text{skip}}$  using layer embeddings:

$$\mathbf{v}_{\text{skip}} = E_{\text{layer}}(l_{t+1}) - E_{\text{layer}}(l_t)$$

This signal encodes the distance and direction of the skip. This signal is then added to the original hidden state for an initial adjustment:

$$h'_t = h_t + \mathbf{v}_{\text{skip}}$$

Finally, the adjusted hidden state  $h'_t$  is passed through a bottleneck-structured Adapter network (an MLP with down-projection and up-projection layers). The final hidden state  $h_{t+1}$ , which is passed to the next layer, is obtained via a residual connection and Layer Normalization:

$$h_{t+1} = \text{LayerNorm}(h_t + \text{MLP}_{\text{adapter}}(h'_t))$$

This design enables the adapter to efficiently learn the non-linear mapping between layer states while preserving the integrity of the original information flow.

### 3.5 Time-aware Importance-weighted Dynamic Reward (TIDR)

The training objective of ASTER is a multi-task composite loss function, designed to synergistically optimize task performance, computational efficiency, and model stability. The total loss,  $\mathcal{L}_{total}$ , is composed of three components:

$$\mathcal{L}_{total} = \lambda_{TASK} \mathcal{L}_{TASK} + \lambda_{RL} \mathcal{L}_{RL} + \lambda_{KD} \mathcal{L}_{KD} \quad (11)$$

where  $\lambda_{TASK}$ ,  $\lambda_{RL}$ , and  $\lambda_{KD}$  are the weighting hyperparameters for their respective loss terms.

The first component is primary task loss, ensuring that the model can still correctly complete downstream tasks (e.g., question answering) after dynamic execution.

The second component is the **Policy Gradient Loss** ( $\mathcal{L}_{RL}$ ). This is the reinforcement learning loss used to optimize the Scorer (the policy network). We employ the REINFORCE algorithm, which aims to maximize the expected discounted cumulative reward. The loss function is defined as:

$$\mathcal{L}_{RL} = -\mathbb{E}_{\tau \sim \pi_{\theta}} \left[ \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | s_t) G_t \right] \quad (12)$$

where  $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$  is the discounted cumulative reward from timestep  $t$ . To obtain a robust reward signal, we designed the **Time-aware Importance-weighted Dynamic Reward (TIDR)**. The TIDR reward,  $r_t$ , is calculated as follows:

$$r_t = \lambda_{eff} \cdot R_{eff} - \lambda_{skip} \cdot P_{skip} + \lambda_{task} \cdot \omega_t \cdot R_{task} \cdot \mathbb{I}(t = T-1) \quad (13)$$

The **Efficiency Reward** ( $R_{eff}$ ) is a dense reward designed to encourage larger layer skips, defined as  $R_{eff} = \log(1 + l_{t+1} - l_t)$ . The **Skip Penalty** ( $P_{skip}$ ) introduces a quadratic penalty to prevent excessively large skips that might harm model performance, formulated as  $P_{skip} = \left( \frac{l_{t+1} - l_t}{L} \right)^2$ . The **Task Reward** ( $R_{task}$ ) is a sparse reward given only at the final step of the decision sequence ( $t = T-1$ ); it is +1 if the final answer is correct, and -1 otherwise. The **Time-aware Importance Weight** ( $\omega_t$ ) assigns a dynamic weight to the sparse task reward, synergistically considering both time decay (earlier decisions have less impact on the final outcome) and skip importance (larger skips have a greater impact). It is defined as  $\omega_t = \left( \frac{T-t}{T} \right) \cdot \sigma \left( \frac{l_{t+1} - l_t}{L} \right)$ , where  $\sigma$  is the Sigmoid function.

### 3.6 Cognition-Based Knowledge Distillation

Transformer models with different architectures, such as BERT and Llama, exhibit fundamental differences in how they carry and process information. Consequently, a "one-size-fits-all" distillation approach is unlikely to achieve optimal results across different models. For encoder models like BERT and DeiT, the core design philosophy revolves around aggregating all information from an input sequence (text or image patches) into a special [CLS] token. The final hidden state of this [CLS] token serves as the "summary"

---

#### Algorithm 1 ASTER: Dynamic Layer Skipping Inference

---

**Require:** LLM layers  $\{\text{Layer}_l\}_{l=0}^{L-1}$ , Policy network  $\pi_{\theta}$  (Scorer), Adapter  $\mathcal{A}$ , Minimum executed layers  $L_{min}$

**Ensure:** Final logits and executed layer path  $P$

```

1:  $H_{in} \leftarrow \text{Embedding}(\text{input})$   $\triangleright$  Initial hidden state from embeddings
2:  $l_{curr} \leftarrow 0$ 
3:  $k \leftarrow 0$   $\triangleright$  Counter for executed layers
4:  $P \leftarrow \emptyset$   $\triangleright$  Path of executed layer indices
5: while  $l_{curr} < L - 1$  do
6:    $H_{out} \leftarrow \text{Layer}_{l_{curr}}(H_{in})$   $\triangleright$  Process the current layer
7:    $P \leftarrow P \cup \{l_{curr}\}$ 
8:    $k \leftarrow k + 1$ 
9:    $h_{cog} \leftarrow \text{ExtractCognitiveState}(H_{out})$   $\triangleright$  e.g., last token's hidden state
10:   $l_{req} \leftarrow \max(0, L_{min} - k)$   $\triangleright$  Calculate required future steps
11:   $l_{max\_allowed} \leftarrow L - l_{req}$ 
12:   $C \leftarrow \{l \mid l \in \{l_{curr} + 1, \dots, L - 1\} \wedge l < l_{max\_allowed}\}$   $\triangleright$  Generate valid candidates
13:  if  $C = \emptyset$  then
14:    break  $\triangleright$  Exit loop if no valid candidates
15:  end if
16:   $\text{scores} \leftarrow \pi_{\theta}(h_{cog}, l_{curr}, C)$ 
17:   $l_{next} \leftarrow \underset{l_j \in C}{\text{argmax scores}[l_j]}$   $\triangleright$  Select highest-scored layer
18:  if  $l_{next} > l_{curr} + 1$  then
19:     $H_{in} \leftarrow \mathcal{A}(H_{out}, l_{curr}, l_{next})$   $\triangleright$  Adapt state for the skip
20:  else
21:     $H_{in} \leftarrow H_{out}$   $\triangleright$  No skip, pass state sequentially
22:  end if
23:   $l_{curr} \leftarrow l_{next}$ 
24: end while
25: for  $l \leftarrow l_{curr}$  to  $L - 1$  do  $\triangleright$  Execute remaining layers sequentially
26:    $H_{out} \leftarrow \text{Layer}_l(H_{in})$ 
27:    $P \leftarrow P \cup \{l\}$ 
28:    $H_{in} \leftarrow H_{out}$ 
29: end for
30:  $\text{logits} \leftarrow \text{LM\_Head}(\text{LayerNorm}(H_{in}))$ 
31: return logits,  $P$ 

```

---

or "feature vector" carefully prepared by the entire model for downstream classification tasks. In contrast, autoregressive decoder models like Llama lack a global aggregation token such as [CLS]. Their primary task is sequence generation, specifically predicting the next token. Therefore, for such models, the definition of "importance" naturally shifts from "contribution to [CLS]" to "contribution to the last meaningful token in the sequence." Mimicking the internal attention distributions that the teacher model relies on when constructing the representation of this final token becomes the most reasonable and direct distillation approach.

Despite their apparent implementation differences, both KD loss formulations can be elegantly unified under a generalized mathematical framework. We define the essence of ASTER's knowledge distillation loss as a task-oriented, importance-weighted feature



regression. Its general form can be expressed as:

$$\mathcal{L}_{KD} = \sum_{i=0}^{N-1} \mathcal{I}(H_S^{(l)}, H_T^{(l+1)}, i) \cdot \text{MSE}(H_{S,i}^{(l)}, H_{T,i}^{(l+1)}) \quad (14)$$

The crux of this framework lies in the Importance Function, we call them  $\mathcal{I}(H_S, H_T, i)$ . This function quantifies the distillation weight for the  $i$ -th token, and its specific formulation is determined by the architecture and task objective of the model being accelerated.

*Specialization for Encoder Architectures (BERT-like):* For encoder models, where the 0-th token is the [CLS] token, the importance function  $\mathcal{I}_{\text{BERT}}$  is defined as a piecewise function to reflect its dual-objective nature:

$$\mathcal{I}_{\text{BERT}}(H_S, H_T, i) = \begin{cases} \lambda_{cls} & \text{if } i = 0 \\ \lambda_{token} \cdot \sigma\left(\frac{H_{T,i} \cdot H_{T,0}}{\tau_{KD}}\right) \cdot \sigma\left(\frac{H_{S,i} \cdot H_{S,0}}{\tau_{KD}}\right) & \text{if } i > 0 \end{cases} \quad (15)$$

The weight for non-[CLS] tokens is determined by the consistency-based attention from both the student and teacher, where a token is deemed important only if both models consider its representation similar to their respective [CLS] representations.  $\lambda_{cls}$  and  $\lambda_{token}$  are conceptual weights for the two objectives.

*Specialization for Autoregressive Architectures (Llama-like):* For autoregressive models, which lack a [CLS] token, the distillation objective is uniform across all tokens. The importance function  $\mathcal{I}_{\text{Llama}}$  is derived directly from the teacher's self-attention distribution towards the final token of the sequence:

$$\mathcal{I}_{\text{Llama}}(H_S, H_T, i) = \lambda_{token} \cdot \left( \text{Softmax} \left( \frac{1}{N_h} \sum_{h=1}^{N_h} A_T^{(l,h)}[:, N] \right) \right)_i \quad (16)$$

In this case, the importance is determined solely by the teacher's internal state, reflecting a direct imitation of the teacher's focus when preparing for next-token prediction. Algorithm 1 demonstrates the Dynamic Layer Skipping Inference process.

## 4 EXPERIMENT

### 4.1 Experimental Setup

**4.1.1 Models.** For the vision modality, we employ Vision Transformer (ViT-Base) [5], a transformer-based architecture with 12 layers, 86M parameters. We also utilize DeiT (Data-efficient Image Transformers) [21], another vision transformer variant with 12 layers, approximately 86M parameters, optimized for efficient training on image data. For the language modality, we utilize Llama 3.1-8B [22], a decoder-only language model with 32 transformer layers and approximately 8 billion parameters. Additionally, we incorporate DistilBERT-base [16], a distilled version of BERT with 6 layers, approximately 66M parameters, designed for efficient natural language processing. All models employ the transformer architecture.

**4.1.2 Datasets.** For vision tasks, we evaluate on ImageNet [4], which contains 1.2M training images and 50K validation images across 1,000 object categories. For language tasks, we use BoolQ [3], a question-answering dataset consisting of 15,942 examples, where each example contains a passage, a question about the passage, and a boolean (yes/no) answer. Additionally, we include the SST-2

dataset [18], a sentiment classification dataset with 67,349 labeled training examples and 1,821 validation examples, focusing on binary positive/negative sentiment.

### 4.2 Baselines

In our experiments, we compare our proposed method with four strong baselines in the field of large language model inference acceleration: Early-Skip, ShortGPT, SkipDecode, and RandomSkip.

**Early-Exit** is an early-exit approach that enables the model to generate outputs from intermediate hidden states without executing a full forward pass. This technique adaptively selects different exit points based on input complexity, thereby reducing the computational resources required for inference. The method incorporates early-exiting structures after specific transformer layers, allowing for faster token generation while maintaining adequate quality.

**ShortGPT** is a model pruning approach that identifies and removes redundant layers in large language models. It defines a metric called Block Influence (BI) to gauge the significance of each layer and directly deletes layers with low BI scores. This method can achieve up to 25% reduction in parameters and computation while maintaining most of the original model's capabilities across various tasks such as reasoning, language understanding, and knowledge retention.

**SkipDecode** is a token-level early exit method specifically designed to work seamlessly with batch inferencing and key-value caching. Unlike other early-exit approaches, SkipDecode sets up a singular exit point for every token in a batch at each sequence position. It guarantees a monotonic decrease in exit points as the sequence progresses, which eliminates the need to recompute key-value caches for preceding tokens. Rather than terminating computation prematurely, it bypasses lower to middle layers, allocating most computational resources to upper layers.

**RandomSkip** serves as a control method in our experiments where layers are randomly skipped during inference based on a predetermined probability distribution.

### 4.3 Main Results

**4.3.1 Comparison with Other Designs.** Table 1 showcases the performance of our method (Ours) on ViT-Base (ImageNet), LLAMA3.1-8B (BoolQ), DeiT (ImageNet), and DistilBERT (SST2) across various acceleration ratios ranging from 1.5× to 3.0×. Our method performs exceptionally well in most scenarios or remains highly competitive with existing baselines, particularly at higher acceleration ratios. Our theoretical acceleration ratio is calculated by dividing the total number of layers in the original model by the average number of layers executed by the ASTER method during inference. This is a theoretical acceleration ratio, not an actual acceleration ratio.

For ViT-Base on ImageNet, the baseline model (using all layers) achieves an accuracy of 81.32%. At 1.5× acceleration, our method reaches 44.46% accuracy, outperforming Early-exit (19.36%), random-skip (20.37%), Skipdecode (33.45%), and Shortgpt (14.39%), leading the second-best method Skipdecode by approximately 11.01 percentage points. As the acceleration ratio increases to 2.0×, our method maintains 32.45% accuracy, while other methods show significant decline, with Shortgpt at only 13.41%. At 3.0× acceleration, our

**Table 1: Comparison of different layer skipping methods across various models and benchmarks. Our method (Ours) consistently outperforms or remains highly competitive with other baselines, especially at higher acceleration rates.**

| Method          | ViT-Base (ImageNet) |               |               | LLAMA3.1-8B (Boolq) |               |               | DeiT (ImageNet) |               |               | DisBERT (SST2) |               |               |
|-----------------|---------------------|---------------|---------------|---------------------|---------------|---------------|-----------------|---------------|---------------|----------------|---------------|---------------|
|                 | 1.5×                | 2.0×          | 3.0×          | 1.14×               | 1.33×         | 2.0×          | 1.71×           | 2.0×          | 3.0×          | 1.2×           | 1.5×          | 3.0×          |
| Baseline (full) | 81.32%              |               |               | 72.48%              |               |               | 81.80%          |               |               | 92.70%         |               |               |
| Early-exit      | 19.36%              | 8.93%         | 2.64%         | 65.35%              | 62.17%        | 57.98%        | 30.42%          | 25.54%        | 17.28%        | <b>90.48%</b>  | 73.97%        | 49.20%        |
| random-skip     | 20.37%              | 0.53%         | 0.15%         | 44.72%              | 0.28%         | 0.01%         | 30.24%          | 14.70%        | 0.80%         | 87.84%         | 80.05%        | 71.56%        |
| Skipdecode      | 33.45%              | 2.64%         | 0.50%         | 45.02%              | 9.05%         | 1.41%         | 21.88%          | 4.64%         | 0.84%         | 88.07%         | 77.06%        | 69.61%        |
| Shortgpt        | 14.39%              | 13.41%        | 3.74%         | <b>71.90%</b>       | 61.33%        | 40.90%        | 54.14%          | 38.10%        | 0.26%         | 88.53%         | 77.41%        | 69.61%        |
| <b>Ours</b>     | <b>44.46%</b>       | <b>32.45%</b> | <b>25.80%</b> | 66.33%              | <b>64.55%</b> | <b>64.50%</b> | <b>74.02%</b>   | <b>65.86%</b> | <b>43.68%</b> | 88.30%         | <b>84.75%</b> | <b>81.65%</b> |

method still sustains 25.80% accuracy, while all other methods fall below 4%.

For LLAMA3.1-8B on BoolQ, the baseline model achieves 72.48% accuracy. At 1.14× acceleration, Shortgpt leads with 71.90% performance, while our method ranks second (66.33%), outperforming Early-exit (62.17%). Additionally, as the acceleration ratio rises to 1.33×, our method (64.55%) surpasses Early-exit (62.17%) and Shortgpt (61.33%). At 2.0× acceleration, our method maintains a stable performance of 64.50%, significantly outperforming the second-best method Early-exit (57.98%).

For DeiT on ImageNet, the baseline model achieves 81.80% accuracy. At 1.71× acceleration, our method achieves 74.02% accuracy, outperforming Early-exit (30.42%), random-skip (30.24%), Skipdecode (21.88%), and Shortgpt (54.14%). At 2.0× and 3.0× acceleration, our method reaches 65.86% and 43.68%, respectively, consistently maintaining the lead.

For DistilBERT on SST2, the baseline model achieves 92.70% accuracy. At 1.2× acceleration, our method reaches 88.30% accuracy, underperforming Early-exit (90.48%) but slightly outperforming random-skip (87.84%), Skipdecode (88.07%), and Shortgpt (88.53%). At 1.5× acceleration, our method sustains 84.75% accuracy, leading Early-exit (73.97%), random-skip (80.05%), and Shortgpt (77.41%). At 3.0× acceleration, our method still maintains 81.65% accuracy, while all other methods exhibit significant declines.

Our experiments reveal distinct degradation patterns across different layer skipping methods. Most existing approaches, such as random-skip and Skipdecode, exhibit a rapid performance decline as the acceleration factor increases. In contrast, our proposed method demonstrates graceful degradation across all acceleration ratios for both model architectures, effectively identifying and preserving critical computational paths, thereby maintaining competitiveness in high-acceleration scenarios.

**4.3.2 Effectiveness of TIDR.** In our analysis, we first disabled the Task-Informed Dynamic Routing (TIDR) mechanism by setting its reward-related parameters to zero, effectively causing the scoring module to generate essentially random scores. This modification resulted in an average of only 4.07 layers being processed on the DeiT-Base model, accompanied by a significant accuracy drop to 13.86%. Second, we compared the full method with simpler baselines that eliminate the scoring model entirely, relying instead on fixed or random path selections. The table below presents the classification accuracy (Acc) for these baselines—Fixed and Random—alongside our proposed method (Ours) across different acceleration ratios

(1.71×, 2.0×, and 3.0×) on the DeiT-Base model. Our method consistently outperforms both baselines, demonstrating the effectiveness of the integrated scoring mechanism.

**Table 2: Comparison of accuracy on DeiT with(out) TIDR.**

| Method         | DeiT (ImageNet) |               |               |
|----------------|-----------------|---------------|---------------|
|                | 1.71x           | 2.0x          | 3.0x          |
| Baseline(1.0x) |                 | 81.80%        |               |
| Fixed          | 7.80%           | 4.12%         | 1.82%         |
| Random         | 9.60%           | 8.92%         | 7.44%         |
| <b>Ours</b>    | <b>74.02%</b>   | <b>65.86%</b> | <b>43.68%</b> |

**4.3.3 Computational and Storage Overhead.** In terms of storage overhead, our proposed Dynamic Adapter is lightweight. When applied to Llama-7B, ASTER introduces an additional storage overhead of approximately 5.42M parameters, which accounts for only 0.077% of the total parameters of the original model. When applied to DistilBERT and ViT, ASTER incurs an additional storage overhead of approximately 0.5M parameters, representing 0.76% of the total parameters of the original models. Regarding computational overhead, for the DeiT-Base model processing 5,000 ImageNet samples, the baseline DeiT-Base model requires 16.87 GFLOPs, whereas our ASTER requires only 5.79 GFLOPs. However, due to the sample-level dynamic layer-skipping strategy, well-established sample batching techniques cannot be extended to our approach. Further optimizing inference latency, for instance, through grouped decision-making, is a key objective for our future work.

## 5 Conclusion

This paper addresses the inefficiency of fixed layer-wise computation in Transformer-based models. We first analyze the growing latency and memory overhead caused by deep architectures and the limitations of static execution. To tackle these issues, we propose ASTER, an adaptive layer-skipping framework that dynamically selects computation paths based on input representations. ASTER employs a scoring model to assess layer importance, applies a minimum path constraint to prevent over-skipping, and introduces dynamic feature representations to retain expressiveness. It is trained with a dual-matching knowledge distillation strategy and a Temporal Importance Difference Reward to balance efficiency and accuracy. Extensive experiments show that ASTER significantly reduces computational costs while maintaining strong predictive performance.



## References

- [1] Zeyuan Allen-Zhu and Yuanzhi Li. 2020. Towards Understanding Ensemble, Knowledge Distillation and Self-Distillation in Deep Learning. *arXiv preprint arXiv:2012.09816* (2020). <https://arxiv.org/abs/2012.09816>
- [2] Luciano Allie Del Giorno. 2023. SkipDecode: Autoregressive Skip Decoding with Batching and Caching for Efficient LLM Inference. *arXiv preprint arXiv:2307.02628* (2023). <https://arxiv.org/abs/2307.02628>
- [3] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the Surprising Difficulty of Natural Questions. *arXiv preprint arXiv:1905.10044* (2019).
- [4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2009), 248–255.
- [5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiuhua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [6] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed Aly, Beidi Chen, and Carole-Jean Wu. 2024. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 12622–12642. doi:10.18653/v1/2024.acl-long.681
- [7] Angela Fan, Edouard Grave, and Armand Joulin. 2019. Reducing Transformer Depth on Demand with Structured Dropout. *arXiv preprint arXiv:1909.11556* (2019). <https://arxiv.org/abs/1909.11556> Referred to as LayerDrop in the text.
- [8] Siqi Fan, Xin Jiang, Xiang Li, Xuying Meng, Peng Han, Shuo Shang, Aixin Sun, Yequan Wang, and Zhongyuan Wang. 2024. Not All Layers of LLMs Are Necessary During Inference. arXiv:2403.02181 [cs.CL] <https://arxiv.org/abs/2403.02181>
- [9] Zhuomin He, Yizhen Yao, Pengfei Zuo, Bin Gao, Qinya Li, Zhenzhe Zheng, and Fan Wu. 2025. AdaSkip: Adaptive Sublayer Skipping for Accelerating Long-Context LLM Inference. arXiv:2501.02336 [cs.CL] <https://arxiv.org/abs/2501.02336>
- [10] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531* (2015). <https://arxiv.org/abs/1503.02531>
- [11] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024. Understanding the planning of LLM agents: A survey. *arXiv preprint arXiv:2402.02716* (2024).
- [12] Ayush Jaiswal, Raunak Jain, Zhe Li, and Yining Yu. 2024. FFN-SkipLLM: A Hidden Gem for Autoregressive Decoding with Adaptive Feed Forward Skipping. *arXiv preprint arXiv:2409.14107* (2024). <https://arxiv.org/abs/2409.14107>
- [13] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. TinyBERT: Distilling BERT for Natural Language Understanding. *Findings of the Association for Computational Linguistics: EMNLP 2020* (2020), 4163–4174. <https://aclanthology.org/2020.findings-emnlp.372>
- [14] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. 2024. ShortGPT: Layers in Large Language Models are More Redundant Than You Expect. arXiv:2403.03853 [cs.CL] <https://arxiv.org/abs/2403.03853>
- [15] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. *arXiv preprint arXiv:1910.01108* (2019). <https://arxiv.org/abs/1910.01108>
- [16] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).
- [17] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident Adaptive Language Modeling. arXiv:2207.07061 [cs.CL] <https://arxiv.org/abs/2207.07061>
- [18] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. 2013. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing* (2013), 1631–1642.
- [19] Siqi Sun, Yu Cheng, Zhe Gan, and Jingjing Liu. 2019. Patient Knowledge Distillation for BERT Model Compression. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (2019), 4323–4332. <https://aclanthology.org/D19-1441>
- [20] Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. 2020. MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (2020), 2158–2170. <https://aclanthology.org/2020.acl-main.195>
- [21] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. 2021. Training data-efficient image transformers & distillation through attention. arXiv:2012.12877 [cs.CV] <https://arxiv.org/abs/2012.12877>
- [22] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).
- [23] Wenhui Wang, Furu Wei, Li Dong, Hang Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. *Advances in Neural Information Processing Systems (NeurIPS)* (2020). <https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [24] Tianwen Wei, Jianwei Qi, and Shenghuan He. 2022. A Flexible Multi-Task Model for BERT Serving. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 785–796. doi:10.18653/v1/2022.acl-short.89
- [25] Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. 2020. BERT Loses Patience: Fast and Robust Inference with Early Exit. arXiv:2006.04152 [cs.CL] <https://arxiv.org/abs/2006.04152>